

AI基础

Lecture 9: Knowledge Representation, Automated Planning, and Quantifying Uncertainty

Yang Shu

School of Data Science and Engineering

yshu@dase.ecnu.edu.cn

[Some slides adapted from Philipp Koehn, JHU]

Lecture 8 ILOs

⋮

- Limitations of propositional logic
- First-order logic (FOL)
- Syntax and Semantics
 - Models, symbols, interpretations, terms
 - Atomic and complex sentences
 - Quantifiers, universal and existential quantifiers, equality
 - Database semantics
- Inference
 - Reduce FOL to propositional logic, instantiation
 - Unification
 - Forward chaining, backward chaining, resolution

$$\text{UNIFY}(p,q) = \theta \text{ where } \text{SUBST}(\theta,p) = \text{SUBST}(\theta,q).$$

Lecture 9 ILOs

- Knowledge representation
 - Ontological engineering
 - Categories of objects, substances, and measures
 - Events
 - Reasoning systems for categories
 - Semantic networks
 - Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Algorithms for classic planning
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Outline

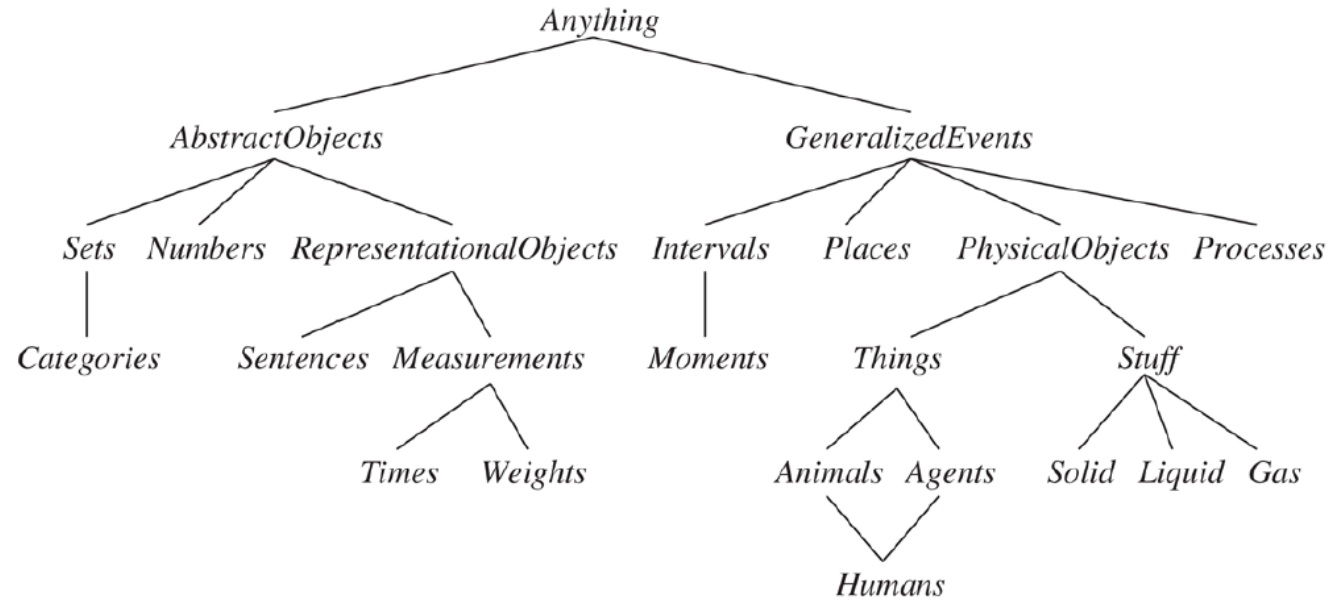
- Knowledge representation
 - Ontological engineering
 - Categories of objects, substances, and measures
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Knowledge representation (知识表示)

- Previous discussions showed how an agent with a knowledge base, either in propositional logic or first-order logic, can make inferences that enable the agent to act appropriately.
- Now we address the question of what content to put into such an agent's knowledge base—how to represent facts about the world.
- Use FOL as the representation language

Ontological engineering (本体论工程)

- General ontology
 - Organize everything in the world into a hierarchy of categories
- The general framework of concepts is called an **upper ontology** because of the convention of drawing graphs with the general concepts at the top and the more specific concepts below them.



The upper ontology of the world, showing the topics to be covered later in the chapter. Each link indicates that the lower concept is a specialization of the upper one. Specializations are not necessarily disjoint—a human is both an animal and an agent. We will see in [Section 10.3.2](#) why physical objects come under generalized events.

Categories (类别)

- The organization of objects into categories is a vital part of knowledge representation.
- Specific objects
 - E.g., my basketball BB_9
- General category, e.g., Basketballs
 - Predicates: $\text{Basketballs}(BB_9)$
 - Reify the category as an object: $\text{Member}(BB_9, \text{Basketballs})$
 - $BB_9 \in \text{Basketball}$
- Subcategories
 - $\text{Basketballs} \subset \text{Balls}$
- Taxonomy
 - System of categories and subcategories
- FOL states facts about categories
 - All members of a category have some properties.
 $(x \in \text{Basketballs}) \Rightarrow \text{Spherical}(x)$
 - Members of a category can be recognized by some properties.
 $\text{Orange}(x) \wedge \text{Round}(x) \wedge \text{Diameter}(x) = 9.5'' \wedge x \in \text{Balls} \Rightarrow x \in \text{Basketballs}$
 - A category as a whole has some properties.
 $\text{Dogs} \in \text{DomesticatedSpecies}$

Basic relations of categories

- Disjoint
- Exhaustive decomposition (完全分解)
- Partition (划分): Exhaustive decomposition of disjoint sets

Disjoint(*{Animals, Vegetables}*)

ExhaustiveDecomposition(*{Americans, Canadians, Mexicans}*,
NorthAmericans)

Partition(*{Animals, Plants, Fungi, Protista, Monera}*,
LivingThings).

$Disjoint(s) \Leftrightarrow (\forall c_1, c_2 \ c_1 \in s \wedge c_2 \in s \wedge c_1 \neq c_2 \Rightarrow Intersection(c_1, c_2) = \{ \})$

$ExhaustiveDecomposition(s, c) \Leftrightarrow (\forall i \ i \in c \Leftrightarrow \exists c_2 \ c_2 \in s \wedge i \in c_2)$

$Partition(s, c) \Leftrightarrow Disjoint(s) \wedge ExhaustiveDecomposition(s, c) .$

Physical composition

- Basic relations such as PartOf

PartOf(Bucharest, Romania)
PartOf(Romania, EasternEurope)
PartOf(EasternEurope, Europe)
PartOf(Europe, Earth).

- Composite objects:
 - A biped is an object with **exactly two** legs attached to a body

$$\begin{aligned} \text{Biped}(a) \Rightarrow \exists l_1, l_2, b \text{ } & \text{Leg}(l_1) \wedge \text{Leg}(l_2) \wedge \text{Body}(b) \wedge \\ & \text{PartOf}(l_1, a) \wedge \text{PartOf}(l_2, a) \wedge \text{PartOf}(b, a) \wedge \\ & \text{Attached}(l_1, b) \wedge \text{Attached}(l_2, b) \wedge \\ & l_1 \neq l_2 \wedge [\forall l_3 \text{ } \text{Leg}(l_3) \wedge \text{PartOf}(l_3, a) \Rightarrow (l_3 = l_1 \vee l_3 = l_2)] . \end{aligned}$$

The notation for “exactly two” is a little awkward; we are forced to say that there are two legs, that they are not the same, and that if anyone proposes a third leg, it must be the same as one of the other two.

Measures (量度)

- In both scientific and commonsense theories of the world, objects have height, mass, cost, and so on. The values that we assign for these properties are called measures.
- We represent the length with a units function that takes a number as argument.

$$\text{Length}(L_1) = \text{Inches}(1.5) = \text{Centimeters}(3.81).$$

$$\text{Centimeters}(2.54 \times d) = \text{Inches}(d).$$

$$\text{Diameter}(\text{Basketball}_{12}) = \text{Inches}(9.5)$$

$$\text{ListPrice}(\text{Basketball}_{12}) = \$ (19)$$

$$\text{Weight}(\text{BunchOf}(\{\text{Apple}_1, \text{Apple}_2, \text{Apple}_3\})) = \text{Pounds}(2)$$

$$d \in \text{Days} \Rightarrow \text{Duration}(d) = \text{Hours}(24).$$

Events (事件)

- So far, facts were treated as true independent of time
- Events: need to describe what is true and when something is happening.
- Event calculus (事件演算)
 - Event, fluents, and time points
- For instance: Flying event

$$E_1 \in \textit{Flyings} \wedge \textit{Flyer}(E_1, \textit{Shankar}) \wedge \textit{Origin}(E_1, \textit{SF}) \wedge \textit{Destination}(E_1, \textit{DC}) .$$

- Flying is the category of all flying events.
 - Reifying events makes it possible to add arbitrary information as attributes
- The event may or may not be ongoing during a specific time
- In general, facts that are true only at specific time points are called fluents, e.g., $\textit{At}(\textit{Shankar}, \textit{Berkeley})$

Predicates of events

$T(f, t_1, t_2)$	Fluent f is true for all times between t_1 and t_2
$Happens(e, t_1, t_2)$	Event e starts at time t_1 and ends at t_2
$Initiates(e, f, t)$	Event e causes fluent f to become true at time t
$Terminates(e, f, t)$	Event e causes fluent f to cease to be true at time t
$Initiated(f, t_1, t_2)$	Fluent f become true at some point between t_1 and t_2
$Terminated(f, t_1, t_2)$	Fluent f cease to be true at some point between t_1 and t_2
$t_1 < t_2$	Time point t_1 occurs before time t_2

- Examples
 - $T(\text{At}(\text{Shankar}, \text{Berkeley}), t_1, t_2)$
 - $Happens(E_1, t_1, t_2)$
- Describe the effects of a flying event

$$E = \text{Flyings}(a, \text{here}, \text{there}) \text{ and } Happens(E, t_1, t_2) \Rightarrow \\ \text{Terminates}(E, \text{At}(a, \text{here}), t_1) \wedge \text{Initiates}(E, \text{At}(a, \text{there}), t_2)$$

Time

- Time points and time intervals
 - Two kinds of time intervals: moments and extended intervals
 - Only moments have zero duration

$$\textit{Partition}(\{\textit{Moments}, \textit{ExtendedIntervals}\}, \textit{Intervals})$$
$$i \in \textit{Moments} \Leftrightarrow \textit{Duration}(i) = \textit{Seconds}(0).$$

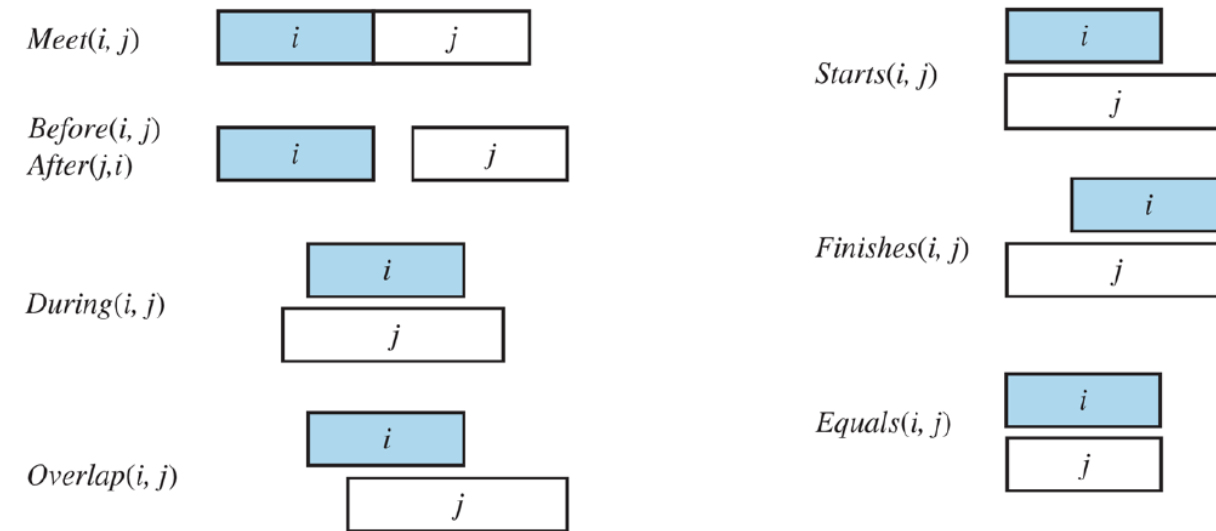
- Absolute time
 - We invent a time scale and associate points on that scale with moments, giving us absolute times.

$$\begin{aligned}\textit{Interval}(i) &\Rightarrow \textit{Duration}(i) = (\textit{Time}(\textit{End}(i)) - \textit{Time}(\textit{Begin}(i))), \\ \textit{Time}(\textit{Begin}(\textit{AD1900})) &= \textit{Seconds}(0). \\ \textit{Time}(\textit{Begin}(\textit{AD2001})) &= \textit{Seconds}(3187324800). \\ \textit{Time}(\textit{End}(\textit{AD2001})) &= \textit{Seconds}(3218860800). \\ \textit{Duration}(\textit{AD2001}) &= \textit{Seconds}(315360000).\end{aligned}$$

Interval relations

$Meet(i, j)$	$\Leftrightarrow End(i) = Begin(j)$
$Before(i, j)$	$\Leftrightarrow End(i) < Begin(j)$
$After(j, i)$	$\Leftrightarrow Before(i, j)$
$During(i, j)$	$\Leftrightarrow Begin(j) < Begin(i) < End(i) < End(j)$
$Overlap(i, j)$	$\Leftrightarrow Begin(i) < Begin(j) < End(i) < End(j)$
$Starts(i, j)$	$\Leftrightarrow Begin(i) = Begin(j)$
$Finishes(i, j)$	$\Leftrightarrow End(i) = End(j)$
$Equals(i, j)$	$\Leftrightarrow Begin(i) = Begin(j) \wedge End(i) = End(j)$

Figure 10.2



Predicates on time intervals.

Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

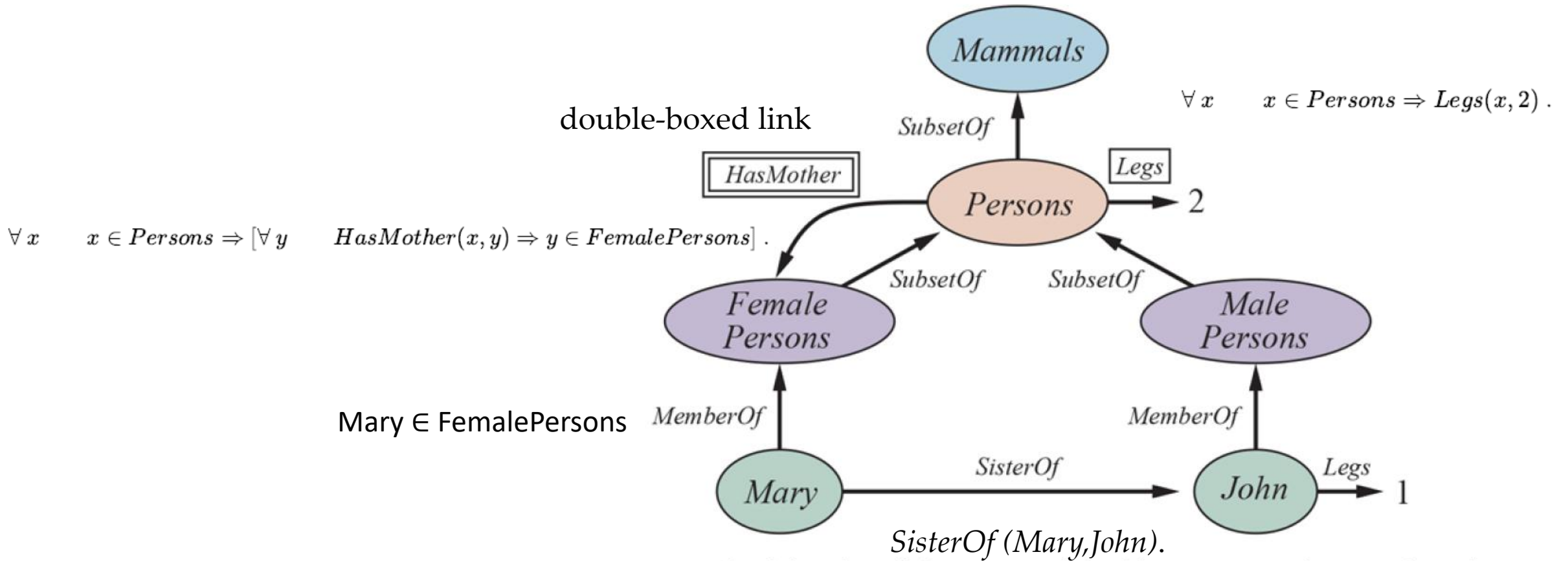
Reasoning systems for categories

- Categories are the primary building blocks of large-scale knowledge representation schemes.
- Systems specially designed for organizing and reasoning with categories. There are two closely related families of systems:
 - **Semantic networks:** provide graphical aids for visualizing a knowledge base and efficient algorithms for inferring properties of an object on the basis of its category membership.
 - **Description logics:** provide a formal language for constructing and combining category definitions and efficient algorithms for deciding subset and superset relationships between categories.

Semantic networks (语义网络)

- There are many variants of semantic networks, but all are capable of representing
 - individual objects, categories of objects, and relations among objects.
- A typical graphical notation displays object or category names in ovals or boxes, and connects them with labeled links.

Figure 10.4

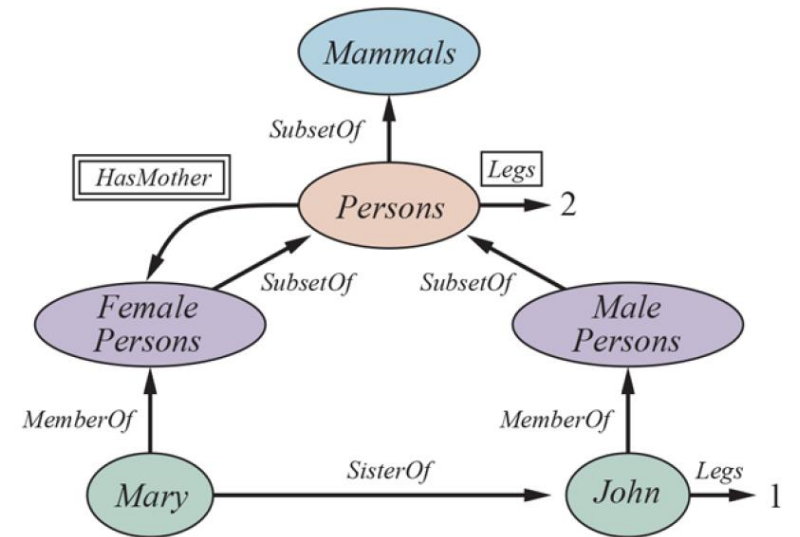


A semantic network with four objects (John, Mary, 1, and 2) and four categories. Relations are denoted by labeled links.

Inheritance (继承) in semantic networks

- The semantic network notation makes it convenient to perform inheritance reasoning.
 - Mary inherits the property of having two legs from Persons.
 - Simplicity and efficiency of the **inheritance** reasoning is one of the main advantageous of semantic networks.
- Multiple inheritance
 - A category can be a subset of more than one other category.
 - May have conflicting values.
 - Often banned in object-oriented programming (OOP). But often allowed in semantic networks.

Figure 10.4



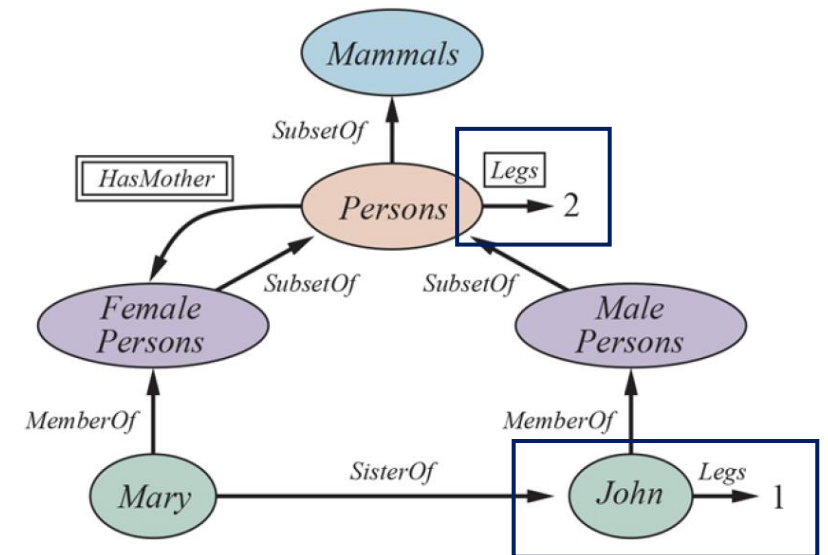
A semantic network with four objects (John, Mary, 1, and 2) and four categories. Relations are denoted by labeled links.

Default values

- One of the most important aspects of semantic networks is their ability to represent default values for categories.
- John has 1 leg but all persons should have two legs.
- In a strictly logical KB, this would be a contradiction.
- In a semantic network, the assertion that all persons have two legs has only default status; that is, a person is assumed to have two legs unless this is contradicted by more specific information.
- Default is **overriden**

$$\forall x \quad x \in \text{Persons} \wedge x \neq \text{John} \Rightarrow \text{Legs}(x, 2) .$$

Figure 10.4



A semantic network with four objects (John, Mary, 1, and 2) and four categories. Relations are denoted by labeled links.

Description logic (描述逻辑)

- The syntax of first-order logic is designed to make it easy to say things about objects.
- Description logics are notations that are designed to make it easier to describe definitions and properties of categories.
- Description logic systems evolved from semantic networks in response to formalize what the networks mean while retaining the emphasis on taxonomic structure as an organizing principle.
- The principal inference tasks for description logics are
 - **Subsumption (包容)** : checking if one category is a subset of another by comparing their definitions
 - **Classification (分类)** : checking whether an object belongs to a category
 - **Consistency (一致性)** of a category definition: whether the membership criteria are logically satisfiable.

The CLASSIC language

- A typical description logic

Bachelor = And(Unmarried, Adult, Male). Categories

Equivalent FOL Objects

Bachelor(x) ⇔ Unmarried(x) ∧ Adult(x) ∧ Male(x) .

To describe the set of men with at least three sons who are all unemployed and married to doctors, and at most two daughters who are all professors in physics or math departments:

*And(Man, AtLeast(3, Son), AtMost(2, Daughter),
 All(Son, And(Unemployed, Married, All(Spouse, Doctor))),
 All(Daughter, And(Professor, Fills(Department, Physics, Math)))).*

Figure 10.6

<i>Concept</i>	→	Thing <i>ConceptName</i>
		And (<i>Concept</i> ,...)
		All (<i>RoleName</i> , <i>Concept</i>)
		AtLeast (<i>Integer</i> , <i>RoleName</i>)
		AtMost (<i>Integer</i> , <i>RoleName</i>)
		Fills (<i>RoleName</i> , <i>IndividualName</i> ,...)
		SameAs (<i>Path</i> , <i>Path</i>)
		OneOf (<i>IndividualName</i> ,...)
<i>Path</i>	→	[<i>RoleName</i> ,...]
<i>ConceptName</i>	→	<i>Adult</i> <i>Female</i> <i>Male</i> ...
<i>RoleName</i>	→	<i>Spouse</i> <i>Daughter</i> <i>Son</i> ...

The syntax of descriptions in a subset of the CLASSIC language.

Outline

- Knowledge representation
 - Ontological engineering
 - Categories of objects, substances, and measures
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Automated Planning (自动规划)

- Factored representation language for planning problems
- Efficient algorithms and heuristics for planning
- Hierarchical actions
- Partially observable and nondeterministic domains
- Scheduling with resource constraints
- Effectiveness

Definition of classical planning

- The task of finding a sequence of actions to accomplish a goal in a discrete, deterministic, static, fully observable environment.
- Existing solutions we learned so far
 - Problem solving agents with search
 - ad hoc heuristics for each new domain: a heuristic evaluation function for search
 - Propositional logical agents
 - moving a step forward had to be repeated for all four agent orientations, T time steps, and n^2 current locations.
- Factored representation with a family of languages called PDDL (Planning Domain Definition Language, 规划领域定义语言)

States (状态)

- A state is represented as a conjunction of ground atomic fluents (基本原子流).
 - “ground” means no variables
 - “fluent” means an aspect of the world that changes over time,
 - “ground atomic” means there is a single predicate, and if there are any arguments, they must be constants, i.e., no variables.
- Examples: States in a package delivery problem
 - $\text{At}(\text{Truck}_1, \text{Melbourne}) \wedge \text{At}(\text{Truck}_2, \text{Sydney})$
 - $\text{At}(x, y)$, not ground
 - $\text{At}(\text{Spouse}(\text{Ali}), \text{Sydney})$, not ground atomic
- PDDL uses database semantics:
 - Unique-names assumption
 - A distinct symbol refers to a distinct object. E.g., two different trucks.
 - Closed world assumption
 - Any fluents that are not mentioned are false

Action (动作)

- An action schema (动作模式)
 - A family of ground actions
- Action name
- A list of variables in the schema
- A precondition (前提):
conjunctions of literals (positive or negated atomic sentences).
- An effect (效果): conjunctions of
literals (positive or negated atomic sentences).

$Action(Fly(p, from, to),$
 $PRECOND: At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$
 $EFFECT: \neg At(p, from) \wedge At(p, to))$

Choose constants to instantiate the variables,
yielding a ground (variable-free) action:

$Action(Fly(P_1, SFO, JFK),$
 $PRECOND: At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK)$
 $EFFECT: \neg At(P_1, SFO) \wedge At(P_1, JFK))$

Applicable actions (适用动作)

- A ground action a is **applicable** in state s if s entails the precondition of a ; that is, if every positive literal in the precondition is in s and every negated literal is not.
- The result of executing applicable action a in state s is defined as a state s'
 - removing the fluents that appear as negative literals in the action's effects (what we call the delete list or $\text{DEL}(a)$),
 - adding the fluents that are positive literals in the action's effects (what we call the add list or $\text{ADD}(a)$):

$$\text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a).$$

Action(Fly(P_1 , SFO, JFK),

PRECOND: $\text{At}(P_1, \text{SFO}) \wedge \text{Plane}(P_1) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$

EFFECT: $\neg \text{At}(P_1, \text{SFO}) \wedge \text{At}(P_1, \text{JFK})$)

State s : $\text{At}(P_1, \text{SFO})$ $\xrightarrow{\text{Action } a: \text{Fly}(P_1, \text{SFO}, \text{JFK})}$ State s' : $\text{At}(P_1, \text{JFK})$

A specific planning problem

- A set of action schemas serves as a definition of a planning domain.
- A specific problem within the domain is defined with the addition of an **initial** state and a **goal**.
- The initial state is a conjunction of ground fluents. As with all states, the closed-world assumption is used, which means that any atoms that are not mentioned are false.
- The goal is a conjunction of literals (positive or negative) that may contain variables.
- For example, goal: $\text{At}(C_1, \text{SFO}) \wedge \neg \text{At}(C_2, \text{SFO}) \wedge \text{At}(p, \text{SFO})$,
 - refers to any state in which cargo C_1 is at SFO but C_2 is not, and in which there is a plane at SFO.

Example domain: Air Cargo Transport

Figure 11.1

Init(*At*(*C*₁, *SFO*) $\wedge$ *At*(*C*₂, *JFK*) $\wedge$ *At*(*P*₁, *SFO*) $\wedge$ *At*(*P*₂, *JFK*)
 $\wedge$ *Cargo*(*C*₁) $\wedge$ *Cargo*(*C*₂) $\wedge$ *Plane*(*P*₁) $\wedge$ *Plane*(*P*₂)
 $\wedge$ *Airport*(*JFK*) $\wedge$ *Airport*(*SFO*))
Goal(*At*(*C*₁, *JFK*) $\wedge$ *At*(*C*₂, *SFO*))

Action(*Load*(*c*, *p*, *a*),
PRECOND: *At*(*c*, *a*) $\wedge$ *At*(*p*, *a*) $\wedge$ *Cargo*(*c*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*a*)
EFFECT: $\neg$ *At*(*c*, *a*) $\wedge$ *In*(*c*, *p*))
Action(*Unload*(*c*, *p*, *a*),
PRECOND: *In*(*c*, *p*) $\wedge$ *At*(*p*, *a*) $\wedge$ *Cargo*(*c*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*a*)
EFFECT: *At*(*c*, *a*) $\wedge$ $\neg$ *In*(*c*, *p*))
Action(*Fly*(*p*, *from*, *to*),
PRECOND: *At*(*p*, *from*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*from*) $\wedge$ *Airport*(*to*)
EFFECT: $\neg$ *At*(*p*, *from*) $\wedge$ *At*(*p*, *to*))

In(*c*, *p*) means that cargo *c* is inside plane *p*
At(*x*, *a*) means that object *x* (either plane or cargo) is at airport *a*.

[*Load*(*C*₁, *P*₁, *SFO*), *Fly*(*P*₁, *SFO*, *JFK*), *Unload*(*C*₁, *P*₁, *JFK*),
Load(*C*₂, *P*₂, *JFK*), *Fly*(*P*₂, *JFK*, *SFO*), *Unload*(*C*₂, *P*₂, *SFO*)].

A PDDL description of an air cargo transportation planning problem.

Example domain: The blocks world

- The domain consists of a set of cube-shaped blocks sitting on an arbitrarily-large table.
- The blocks can be stacked, but only one block can fit directly on top of another.
- A robot arm can pick up a block and move it to another position, either on the table or on top of another block. The arm can pick up only one block at a time, so it cannot pick up a block that has another one on top of it.

Figure 11.3

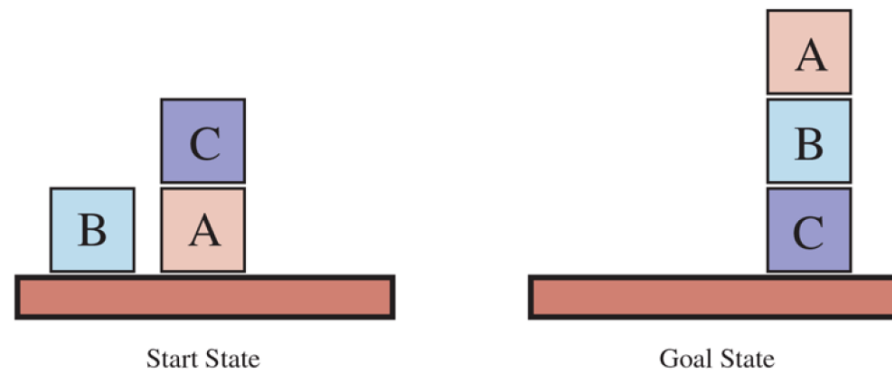


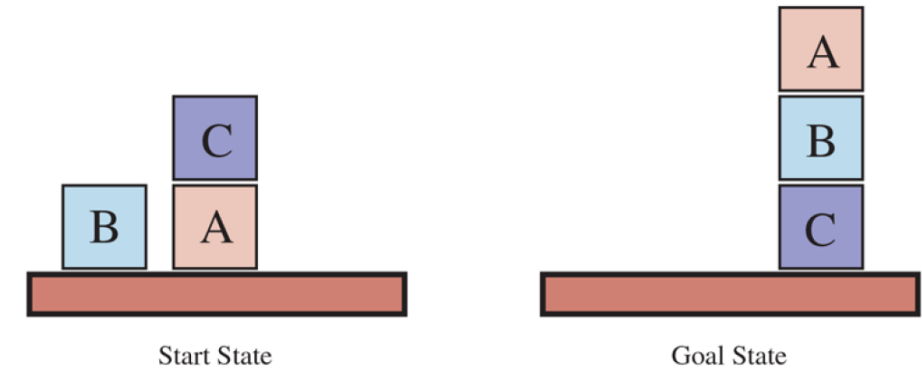
Diagram of the blocks-world problem in Figure 11.4.

Figure 11.4

arbitrarily-large table

$Init(On(A, Table) \wedge On(B, Table) \wedge On(C, A)$
 $\wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C) \wedge Clear(Table))$
 $Goal(On(A, B) \wedge On(B, C))$
 $Action(Move(b, x, y),$
 $\quad PRECOND: On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$
 $\quad (b \neq x) \wedge (b \neq y) \wedge (x \neq y),$
 $\quad EFFECT: On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$
 $Action(MoveToTable(b, x),$
 $\quad PRECOND: On(b, x) \wedge Clear(b) \wedge Block(b) \wedge Block(x),$
 $\quad EFFECT: On(b, Table) \wedge Clear(x) \wedge \neg On(b, x))$

A planning problem in the blocks world: building a three-block tower. One solution is the sequence $[MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)]$.

Diagram of the blocks-world problem in [Figure 11.4](#).

- $Move(b, x, y)$: move block b from x to y
- predicate $clear(x)$ that is true when nothing is on x .

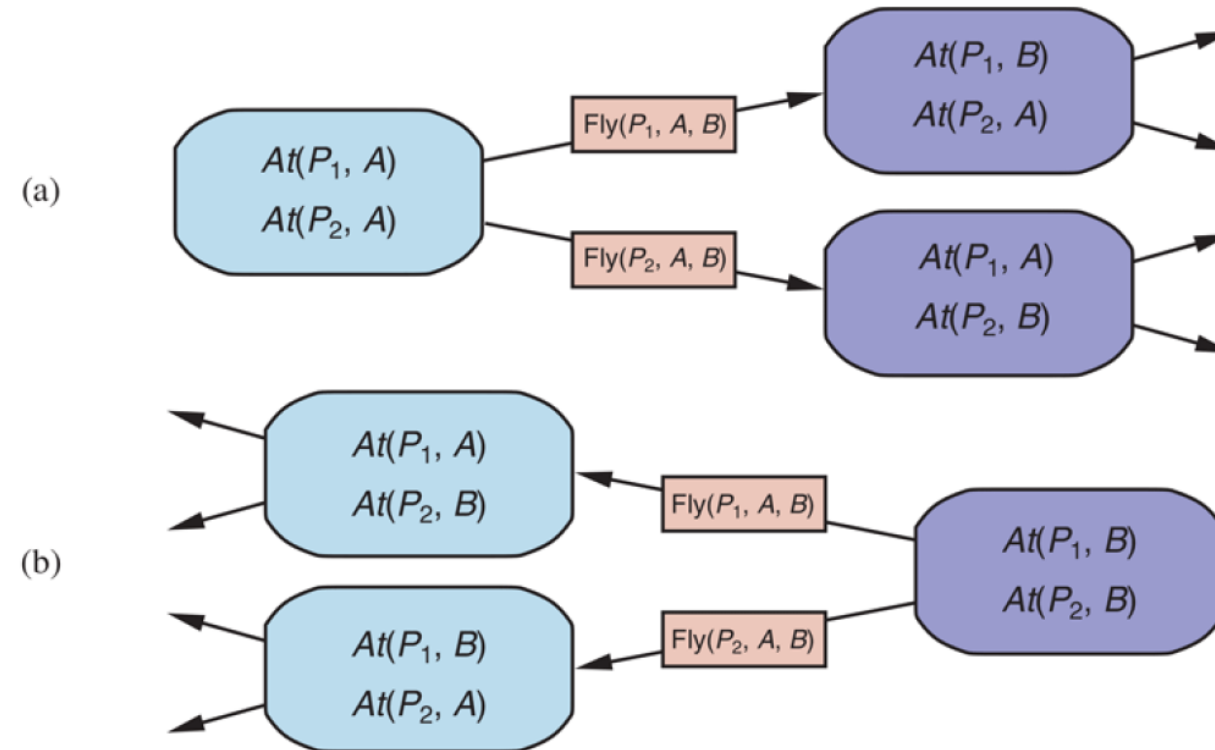
Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Algorithms for classical planning

- Three possible ways
 - The description of a planning problem provides an obvious way to **search** from the initial state through the space of states, looking for a goal.
 - A nice advantage of the declarative representation of action schemas is that we can also **search backward** from the goal, looking for the initial state.
 - A third possibility is to translate the problem description into a set of logic sentences, to which we can apply a **logical inference** algorithm to find a solution.

Figure 11.5

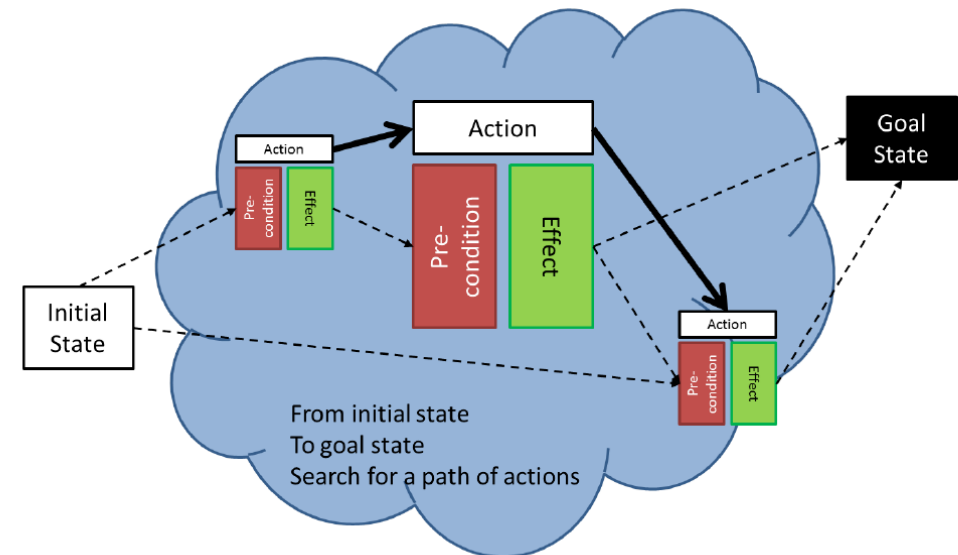


Two approaches to searching for a plan. (a) Forward (progression) search through the space of ground states, starting in the initial state and using the problem's actions to search forward for a member of the set of goal states. (b) Backward (regression) search through state descriptions, starting at the goal and using the inverse of the actions to search backward for the initial state.

Forward state-space search

- Search algorithms
- The states in this search state space are ground states, where every fluent is either true or not.
- The goal is a state that has all the positive fluents in the problem's goal and none of the negative fluents.
- State-space search considers actions that are **applicable**

A ground action a is **applicable** in state s if s entails the precondition of a ; that is, if every positive literal in the precondition is in s and every negated literal is not.



Backward search

- In backward search (also called regression search) we start at the goal and apply the actions backward until we find a sequence of steps that reaches the initial state.
- At each step we consider **relevant** actions (in contrast to forward search, which considers actions that are applicable).
- This reduces the branching factor significantly, particularly in domains with many possible actions.
- A **relevant action** is one with an effect that unifies with one of the goal literals, but with no effect that negates any part of the goal.
 - With the goal $\neg \text{Poor} \wedge \text{Famous}$, an action with the sole effect Famous would be relevant, but one with the effect $\text{Poor} \wedge \text{Famous}$ is not considered relevant: because then Poor would appear in the final state.

Other planning approaches

- Planning as **Boolean satisfiability**
 - Translating a PDDL problem description into propositional form.
- Graphplan uses a **planning graph** to encode constraints on how actions are related to their preconditions and effects.
- **Situation calculus** describes planning problems in FOL.
 - Has not made a big impact in practical applications
- **Partial-order planning**
 - each action is a node in the graph, and for each precondition of the action there is an edge from another action (or from the initial state) that indicates that the predecessor action establishes the precondition.
 - We search in the space of plans rather than world-states, inserting actions to satisfy conditions.
 - Often used in domains where it is important for humans to understand the plans.

Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Heuristics for planning

- Neither forward nor backward search is efficient without a good heuristic function.
- Recall that a heuristic function $h(s)$ estimates the distance from a state to the goal, and that if we can derive an admissible heuristic for this distance—one that does not overestimate—then we can use A* search to find optimal solutions.
- By definition, there is no way to analyze an atomic state, and thus it requires some ingenuity by an analyst (usually human) to define good **domain-specific** heuristics for search problems with atomic states.
 - Generating heuristics from relaxed problems
 - Generating heuristics from subproblems, pattern databases
 - Generating heuristics from with landmarks
 - Learning heuristics from experience
- But planning uses a factored representation for states and actions, which makes it possible to define good **domain-independent** heuristics.

Relaxed problem

- Recall that an admissible heuristic can be derived by defining a relaxed problem that is easier to solve.
- The exact cost of a solution to this easier problem then becomes the heuristic for the original problem.
- A search problem is a graph where the nodes are states and the edges are actions. The problem is to find a path connecting the initial state to a goal state.
- There are two main ways we can relax this problem to make it easier:
 - By **adding more edges** to the graph, making it strictly easier to find a path; (adding short-cuts)
 - By **grouping multiple nodes together**, forming an abstraction of the state space that has fewer states, and thus is easier to search.

Adding edges, ignore-preconditions heuristics

- Drops all preconditions from actions.
- Every action becomes applicable in every state, and any single goal fluent can be achieved in one step (if there are any applicable actions—if not, the problem is impossible).
- This almost implies that the number of steps required to solve the relaxed problem is the number of unsatisfied goals
 - Almost but not quite, because (1) some action may achieve multiple goals and (2) some actions may undo the effects of others.

Ignore-preconditions heuristics

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- We relax the actions by removing all preconditions and all effects except those that are literals in the goal.
- Then, we count the minimum number of actions required such that the union of those actions' effects satisfies the goal.
 - Set-cover problem
- It is also possible to ignore only selected preconditions of actions.

Action(Slide (t, s₁, s₂),

PRECOND:On (t, s₁) ∧ Tile (t) ∧ Blank (s₂) ∧ Adjacent (s₁, s₂),

EFFECT:On (t, s₂) ∧ Blanks (s₁) ∧ ¬On (t, s₁) ∧ ¬Blank (s₂))

- Remove Blank(s₂) and Adjacent(s₁, s₂): number of misplaced tiles
- Remove Blank(s₂): Manhattan distance

Ignore delete-lists heuristic

- Assume that all goals and preconditions contain only positive literals.
 - replace every negative literal $\neg P$ in a goal or precondition with a new positive literal p' , and modify the initial state and the action effects accordingly.
- We want to create a relaxed version of the original problem that will be easier to solve, and where **the length** of the solution will serve as a good heuristic.
- We can do that by removing the delete lists from all actions (i.e., removing all negative literals from effects).

Reducing states: State abstraction

- Relaxing the actions does nothing to reduce the number of states, which means that it may still be expensive to compute the heuristic.
- We can look at relaxations that decrease the number of states by forming a state abstraction—a many-to-one mapping from states in the ground representation of the problem to the abstract representation.

Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Hierarchical planning

- A high-level plan for a Hawaii vacation might be “Go to San Francisco airport; take flight HA 11 to Honolulu; do vacation stuff for two weeks; take HA 12 back to San Francisco; go home.”
- Given such a plan, the action “Go to San Francisco airport” can be viewed as a planning task in itself, with a solution such as “Choose a ride-hailing service; order a car; ride to airport.”
- Each of these actions, in turn, can be decomposed further, until we reach the low-level motor control actions like a button-press.
- **Hierarchical decomposition (层次分解)**: an idea that pervades almost all attempts to manage complexity.

Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Planning and Acting in Nondeterministic Domains

- **Sensorless planning** (also known as conformant planning) for environments with no observations;
 - Belief state planning
- **Contingency planning** for partially observable and nondeterministic environments;
 - AND-OR search
- **Online planning** and replanning for unknown environments.

Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Time, Schedules, and Resources

- Classical planning talks about what to do, in what order, but does not talk about time: how long an action takes and when it occurs.
- For example, in the airport domain we could produce a plan saying what planes go where, carrying what, but could not specify departure and arrival times. This is the subject matter of **scheduling**.
- The real world also imposes resource constraints: an airline has a limited number of staff, and staff who are on one flight cannot be on another at the same time.
 - Techniques for planning and scheduling problems with resource constraints.
- The approach we take is “**plan first, schedule later**”: divide the overall problem into
 - A planning phase in which actions are selected, with some ordering constraints, to meet the goals of the problem, and
 - A scheduling phase, in which temporal information is added to the plan to ensure that it meets resource and deadline constraints.
- Solutions can be evaluated according to a cost function
 - For simplicity, cost function is just the total duration of the plan, called the makespan.

Representing temporal and resource constraints

- A typical job-shop scheduling problem, consists of a set of **jobs**, each of which has a collection of **actions** with ordering constraints among them.
- Each action has a duration and a set of resource constraints required by the action. A constraint
 - specifies a type of resource (e.g., bolts, wrenches, or pilots)
 - the number of that resource required
 - whether that resource is consumable (e.g., the bolts are no longer available for use) or reusable (e.g., a pilot is occupied during a flight but is available again when the flight is over).
- Actions can also produce resources (e.g., manufacturing and resupply actions).

Example: the assembly of two cars

- The representation of resources as numerical quantities, such as Inspectors(2), rather than as named entities, such as I_1 and I_2 , is an example of a technique called **aggregation**.
- Aggregation is essential for reducing complexity.
 - Consider what happens when a proposed schedule has 10 concurrent Inspect actions but only 9 inspectors are available. With inspectors represented as quantities, a failure is detected immediately and the algorithm backtracks to try another schedule.
 - With inspectors represented as individuals, the algorithm would try all $9!$ ways of assigning inspectors to actions before noticing that none of them work.

Figure 11.13

```
Jobs({AddEngine1  $\prec$  AddWheels1  $\prec$  Inspect1},
      {AddEngine2  $\prec$  AddWheels2  $\prec$  Inspect2})

Resources(EngineHoists(1), WheelStations(1), Inspectors(e2), LugNuts(500))

Action(AddEngine1, DURATION:30,
       USE:EngineHoists(1))
Action(AddEngine2, DURATION:60,
       USE:EngineHoists(1))
Action(AddWheels1, DURATION:30,
       CONSUME:LugNuts(20), USE:WheelStations(1))
Action(AddWheels2, DURATION:15,
       CONSUME:LugNuts(20), USE:WheelStations(1))
Action(Inspecti, DURATION:10,
       USE:Inspectors(1))
```

reusable

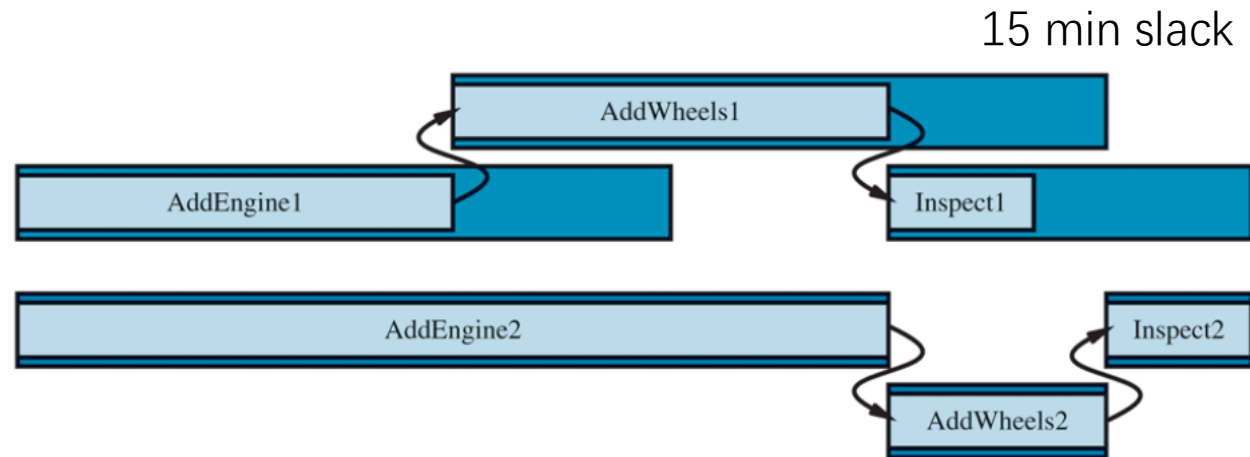
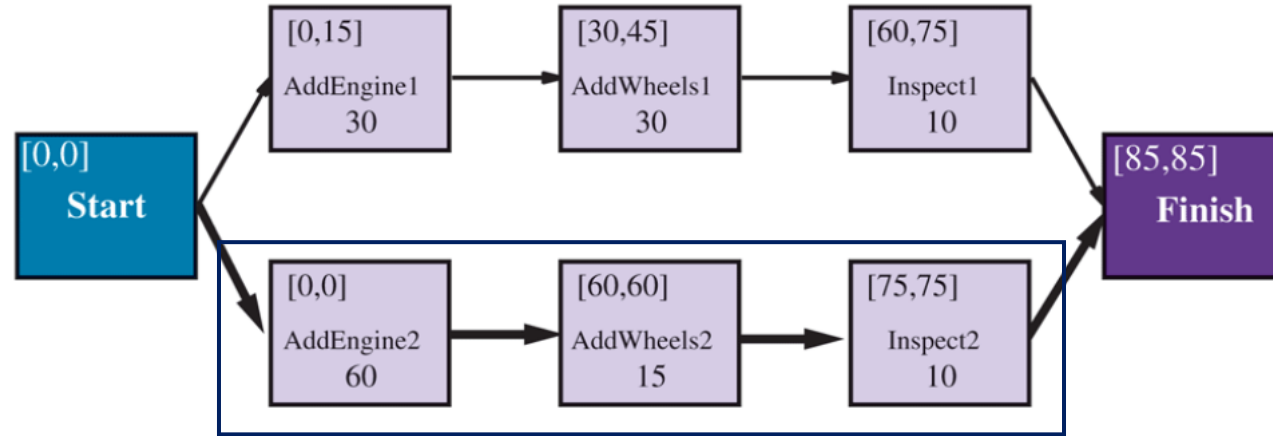
consumable

A job-shop scheduling problem for assembling two cars, with resource constraints. The notation $A \prec B$ means that action A must precede action B .

Solving scheduling problem

- For now, considering just the temporal scheduling problem, but ignoring resource constraints.
- To minimize makespan (plan duration), we must find the **earliest start times** for all the actions consistent with the ordering constraints supplied with the problem. It is helpful to view these ordering constraints as a **directed graph** relating the actions.
- We can apply the critical path method (CPM) to this graph to determine the possible start and end times of each action. A path through a graph representing a partial-order plan is a linearly ordered sequence of actions beginning with Start and ending with Finish.

- **The critical path** is that path whose total duration is longest; the path is “critical” because it determines the duration of the entire plan—shortening other paths doesn’t shorten the plan as a whole, but delaying the start of any action on the critical path slows down the whole plan.
- Actions that are off the critical path have a window of time in which they can be executed. The window is specified in terms of an earliest possible start time, ES, and a latest possible start time, LS. The quantity $LS - ES$ is known as the slack of an action.
- We can see that the whole plan will take 85 minutes, that each action in the top job has 15 minutes of slack, and that each action on the critical path has no slack (by definition).
- Together the ES and LS times for all the actions constitute a schedule for the problem.



Defining ES and LS

A and B are actions, and $A \prec B$ means that A precedes B :

$$ES(Start) = 0$$

$$ES(B) = \max_{A \prec B} ES(A) + Duration(A)$$

$$LS(Finish) = ES(Finish)$$

$$LS(A) = \min_{A \prec B} LS(B) - Duration(A).$$

The maximum of **the earliest finish times** of those immediately preceding actions.

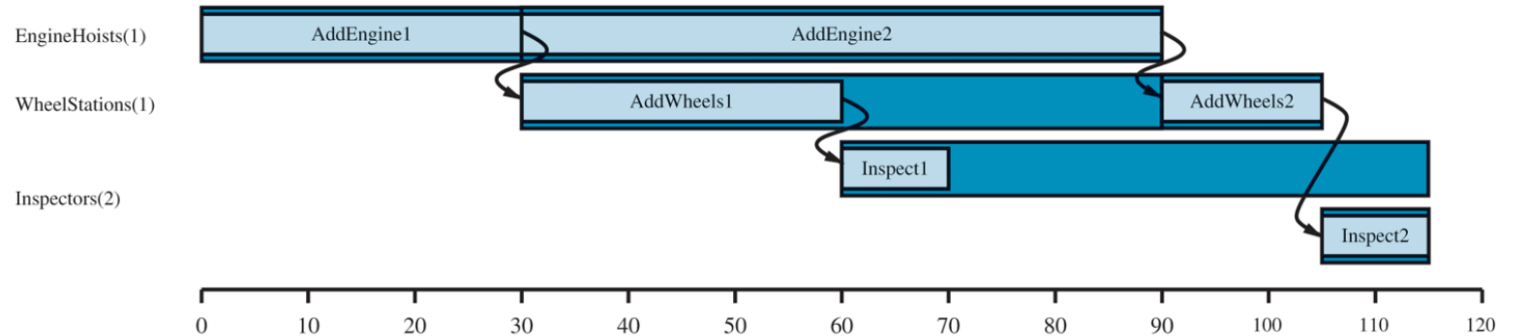
The earliest finish time = the earliest start time + the duration.

Often use dynamic-programming algorithms

The complexity of the critical path algorithm is just $O(Nb)$, where N is the number of actions and b is the maximum branching factor into or out of an action. (To see this, note that the LS and ES computations are done once for each action, and each computation iterates over at most b other actions.) Therefore, finding a minimum-duration schedule, given a partial ordering on the actions and no resource constraints, is quite easy.

Resource constraints

- More complicated



- One popular heuristic is the **minimum slack** heuristic: on each iteration, schedule for the earliest possible start whichever unscheduled action has all its predecessors scheduled and has the least slack.
 - The heuristic often works well in practice. But in our example,
 - AddEngine2 will be selected first, thus resulting in a 130-min solution
 - Optimal solution: 115-minute

Analysis of Planning Approaches

- Planning combines the two major areas of AI we have covered so far: search and logic.
- A planner can be seen either as a program that searches for a solution or as one that (constructively) proves the existence of a solution.
- Unfortunately, we do not yet have a clear understanding of which techniques work best on which kinds of problems.
- New techniques will emerge, perhaps providing a synthesis of highly expressive first-order and hierarchical representations with the highly efficient factored and propositional representations that dominate today.
- Portfolio planning systems, where a collection of algorithms are available to apply to any given problem.
 - This can be done selectively (the system classifies each new problem to choose the best algorithm for it).
 - or in parallel (all the algorithms run concurrently, each on a different CPU).
 - or by interleaving the algorithms according to a schedule.

Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Acting under uncertainty

- Agents in the real world need to handle uncertainty, whether due to partial observability, nondeterminism, or adversaries.
- Let action A_t = leave for airport t minutes before flight.
 - Will A_t get me there on time?
- Problems
 - partial observability (road state, other drivers' plans, etc.)
 - noisy sensors (traffic reports)
 - uncertainty in action outcomes (flat tire, etc.)
 - immense complexity of modelling and predicting traffic
- Hence a purely logical approach either
 - risks falsehood: " A_{90} will get me there on time"
 - leads to conclusions that are too weak for decision making:
 - " A_{90} will get me there on time if there's no accident on the bridge and it doesn't rain and my tires remain intact, and no meteorite hits the car, and..."
 - None of these conditions can be deduced for sure, so we can't infer that the plan succeeds.

Toothache $\Rightarrow$ *Cavity*.

Methods for handling uncertainty *Toothache* $\Rightarrow$ *Cavity* $\vee$ *GumProblem* $\vee$ *Abscess*...

- Logic fails to cope uncertainty
 - Laziness: It is too much work to list the complete set of antecedents or consequents needed to ensure an exceptionless rule and too hard to use such rules.
 - Ignorance:
 - Theoretical ignorance: Medical science has no complete theory for the domain.
 - Practical ignorance: Even if we know all the rules, we might be uncertain about a particular patient because not all the necessary tests have been or can be run.
- The theory of probability provides a way of summarizing the uncertainty that comes from our laziness and ignorance, thereby solving the qualification problem.

Uncertainty and rational decisions (理性决策)

$$P(A_{25} \text{ gets me there on time} | \dots) = 0.04$$

$$P(A_{90} \text{ gets me there on time} | \dots) = 0.70$$

$$P(A_{120} \text{ gets me there on time} | \dots) = 0.95$$

$$P(A_{1440} \text{ gets me there on time} | \dots) = 0.9999$$

- Consider again the plan A_{90} for getting to the airport.
- Suppose A_{90} gives us a 70% chance of catching our flight.
- Does this mean it is a rational choice?
 - Not necessarily: there might be other plans, such as A_{120} , with higher probabilities.
 - If it is vital not to miss the flight, then it is worth risking the longer wait at the airport.
- What about A_{1440} , a plan that involves leaving home 24 hours in advance?
 - In most circumstances, this is not a good choice, because although it almost guarantees getting there on time, it involves an intolerable wait—not to mention a possibly unpleasant diet of airport food.

Preferences

- To make such choices, an agent must first have preferences among the different possible outcomes of the various plans. An outcome is a completely specified state, including such factors as whether the agent arrives on time and the length of the wait at the airport.
- We use utility theory to represent preferences and reason quantitatively with them. (The term utility is used here in the sense of “the quality of being useful.”)
- Utility theory says that every state (or state sequence) has a degree of usefulness, or utility, to an agent and that the agent will prefer states with higher utility.
- Preferences, as expressed by utilities, are combined with probabilities in the general theory of rational decisions called **decision theory**
 - **Decision theory = probability theory + utility theory.**

Rational agent

Jilin Hu, Bin Yang, Chenjuan Guo, Christian S. Jensen: Risk-aware path selection with time-varying, uncertain travel costs: a time series approach. VLDB J. 27(2): 179-200 (2018)

- The fundamental idea of **decision theory** is that an agent is rational if and only if it chooses the action that yields the highest expected utility, averaged over all the possible outcomes of the action.
- This is called the principle of **maximum expected utility** (MEU).

Table 1 Uncertain travel times for P_1 , P_2 , and P_3 , [8:00, 8:15)

Travel time (min)	70	80	90	100	110	120
P_1	0	0.25	0.50	0	0	0.25
P_2	0	0	0.50	0.50	0	0
P_3	0	0	0	0.50	0	0.50

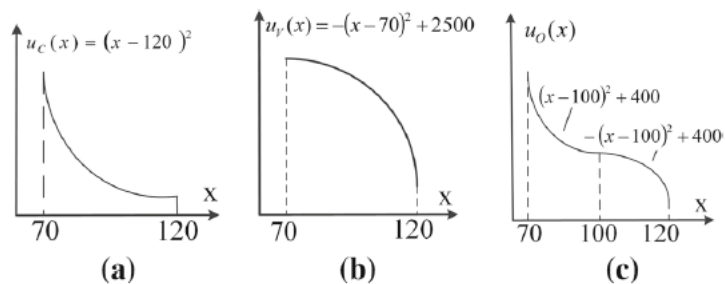


Fig. 2 Categorization of utility functions. a Convex, b concave and c other

Table 3 Risk preferences, utility functions, and stochastic dominance

Risk attitudes	Utility functions	Stochastic dominance
Risk-neutral	Non-increasing	First order
Risk-loving	Non-incr., convex	Second convex order
Risk-averse	Non-incr., concave	Second concave order

Table 4 Expected utilities for paths P_1 , P_2 , and P_3

	P_1	P_2	P_3	Optimal
u_C	850	650	200	P_1
u_V	1650	1850	800	P_2
u_O	450	450	200	P_1, P_2

Outline

- Knowledge representation
 - Events
 - Reasoning systems for categories: Semantic networks, Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities

Probability theory

- Begin with a set Ω —the sample space
e.g., 6 possible rolls of a die.
 $\omega \in \Omega$ is a sample point/possible world/atomic event■
- A probability space or probability model is a sample space with an assignment $P(\omega)$ for every $\omega \in \Omega$ s.t.
 $0 \leq P(\omega) \leq 1$
 $\sum_{\omega} P(\omega) = 1$
e.g., $P(1) = P(2) = P(3) = P(4) = P(5) = P(6) = 1/6$.■
- An event A is any subset of Ω

$$P(A) = \sum_{\{\omega \in A\}} P(\omega)$$

- E.g., $P(\text{die roll} \leq 3) = P(1) + P(2) + P(3) = 1/6 + 1/6 + 1/6 = 1/2$

- In logic, a set of worlds corresponds to a proposition in a formal language.
 - For each proposition, the corresponding set contains just those possible worlds in which the proposition holds.

$$\text{For any proposition } \phi, P(\phi) = \sum_{\omega \in \phi} P(\omega).$$

- Rolling two fair dice
 - $P(\text{Total}=11) = P((5, 6)) + P((6, 5)) = 1/36 + 1/36 = 1/18$

Priors and posterior

- Unconditional/prior probability, priors
 - Degrees of belief in propositions **in the absence of any other information**
- Condition/posterior probability, posterior, denoted $P(a|b)$
 - Total is 11, given the first die is a 5.

$$P(a|b) = \frac{P(a \wedge b)}{P(b)}, \quad P(a \wedge b) = P(a|b)P(b).$$

- $P(\text{total}=11|5) = P(\text{total}=11 \text{ and one die is } 5) / P(\text{one die is } 5) = (1/36) / (1/6) = 6/36 = 1/6$
- $P(\text{Total}=11) = P((5, 6)) + P((6,5)) = 1/36 + 1/36 = 1/18$

Random variables

- Variables in probability theory are called random variables.

- The range of Total for two dice is the set $\{2, 3, \dots, 12\}$

- Weather = {sun, rain, cloud, snow}

- Probability distribution $\mathbf{P}(\text{Weather}) = \langle 0.6, 0.1, 0.29, 0.01 \rangle$

- categorical distribution, probability mass function (pmf)

$$P(\text{Weather} = \text{sun}) = 0.6$$

$$P(\text{Weather} = \text{rain}) = 0.1$$

$$P(\text{Weather} = \text{cloud}) = 0.29$$

$$P(\text{Weather} = \text{snow}) = 0.01,$$

- For continuous variables, probability density function (pdf)

$$P(\text{NoonTemp} = x) = \text{Uniform}(x; 18C, 26C) = \begin{cases} \frac{1}{8C} & \text{if } 18C \leq x \leq 26C \\ 0 & \text{otherwise} \end{cases}$$

Distributions on multiple variables

- Joint probability function
- Weather={sun, rain, cloud, snow}
- Cavity={true, false}
- $P(\text{Weather}, \text{Cavity})$, $4 \times 2 = 8$ possible worlds.

$$\mathbf{P}(\text{Weather}, \text{Cavity}) = \mathbf{P}(\text{Weather} \mid \text{Cavity})\mathbf{P}(\text{Cavity}),$$

$$\begin{aligned}P(W = \text{sun} \wedge C = \text{true}) &= P(W = \text{sun} \mid C = \text{true}) P(C = \text{true}) \\P(W = \text{rain} \wedge C = \text{true}) &= P(W = \text{rain} \mid C = \text{true}) P(C = \text{true}) \\P(W = \text{cloud} \wedge C = \text{true}) &= P(W = \text{cloud} \mid C = \text{true}) P(C = \text{true}) \\P(W = \text{snow} \wedge C = \text{true}) &= P(W = \text{snow} \mid C = \text{true}) P(C = \text{true}) \\P(W = \text{sun} \wedge C = \text{false}) &= P(W = \text{sun} \mid C = \text{false}) P(C = \text{false}) \\P(W = \text{rain} \wedge C = \text{false}) &= P(W = \text{rain} \mid C = \text{false}) P(C = \text{false}) \\P(W = \text{cloud} \wedge C = \text{false}) &= P(W = \text{cloud} \mid C = \text{false}) P(C = \text{false}) \\P(W = \text{snow} \wedge C = \text{false}) &= P(W = \text{snow} \mid C = \text{false}) P(C = \text{false}).\end{aligned}$$

Lecture 9 ILOs

- Knowledge representation
 - Ontological engineering
 - Categories of objects, substances, and measures
 - Events
 - Reasoning systems for categories
 - Semantic networks
 - Description logic
- Automated planning
 - Classic planning and Planning Domain Definition Language (PDDL)
 - Algorithms for classic planning
 - Forward search (progression) and backward search (regression)
 - Heuristics: adding shortcut edges, reducing states
 - Hierarchical planning
 - Planning in nondeterministic domains
 - Scheduling
- Quantifying uncertainty
 - Acting under uncertainty
 - Probabilities